

Lab6 Conditional DDPM for Image Generation Report

Deep Learning, NYCU

朱驛庭, 111550093

1 Introduction

The objective of this lab is to implement a conditional generative model that synthesizes iCLEVR images from multi-label conditions. Each condition contains up to three objects, where every object is defined by a color-shape pair such as “red sphere” or “cyan cylinder”. The generated images are evaluated by a fixed pretrained ResNet18 classifier supplied by the course staff.

The implemented system follows the DDPM formulation and uses a conditional UNet to denoise images according to multi-label object conditions. To improve controllability, I combine three guidance mechanisms: label conditioning inside the UNet, classifier-free guidance during sampling, and classifier guidance from the pretrained evaluator. The final model (v5, no center crop) achieves **0.9722** accuracy on `test.json` and **0.9524** accuracy on `new_test.json`.

2 Implementation Details

2.1 Dataset and Preprocessing

The training set contains 18,009 images and JSON annotations. Labels are converted into 24-dimensional multi-hot vectors according to `objects.json`. The raw images are RGBA images with size 320×240 , so I convert each image to RGB before applying transforms.

The final training transform is:

1. convert RGBA to RGB,
2. resize directly to 64×64 (square resize, no center crop),
3. random horizontal flip for augmentation,
4. normalize with mean and standard deviation (0.5, 0.5, 0.5).

The same normalization is used for evaluator input, as required by the specification. Generated images are denormalized only when saving PNG files. Earlier experiments (v1–v4) used `CenterCrop(240)` followed by `Resize(64)` to preserve aspect ratio, but this discards roughly 25% of the horizontal field of view in the original 320×240 frames and removes objects placed near the left or right border. The final model (v5) drops the crop and resizes the full frame directly to 64×64 , which exposes the model to all object positions and significantly improves both training-time and post-hoc evaluator accuracy.

The core dataset implementation loads image filenames and label lists from `train.json`, then converts the labels into

multi-hot tensors. The same encoder is reused for `test.json` and `new_test.json` so that the evaluator receives labels in the required format.

```
1 def encode_labels(label_lists, obj2idx,
2   ↪ num_classes=24):
3     out = torch.zeros(len(label_lists),
4   ↪ num_classes)
5     for i, names in enumerate(label_lists):
6         for name in names:
7             out[i, obj2idx[name]] = 1.0
8     return out
9
10 transform = transforms.Compose([
11     transforms.Lambda(lambda im:
12   ↪ im.convert("RGB")),
13     transforms.Resize((64, 64), antialias=True),
14     transforms.RandomHorizontalFlip(),
15     transforms.ToTensor(),
16     transforms.Normalize((0.5,)*3, (0.5,)*3),
17 ])
```

2.2 Conditional DDPM Formulation

The model is trained with the standard DDPM noise prediction objective. Given a clean image x_0 , a sampled timestep t , and Gaussian noise ϵ , the forward process creates

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon.$$

The UNet predicts $\epsilon_\theta(x_t, t, y)$, where y is the multi-hot condition vector. The training loss is

$$\mathcal{L}_{\text{DDPM}} = \|\epsilon - \epsilon_\theta(x_t, t, y)\|_2^2.$$

I use 1,000 diffusion timesteps and the cosine beta schedule `squaredcos_cap_v2`. The model is trained with epsilon prediction. During evaluation, I use DDIM sampling for faster generation, with 100 inference steps in the final setting.

The scheduler configuration is intentionally simple and shared between training and inference. Training uses DDPM, while inference uses DDIM with the same cumulative noise schedule.

```
1 from diffusers import DDIMScheduler, DDPMScheduler
2
3
4 def build_train_scheduler(num_train_timesteps):
5     return DDPMScheduler(
6         num_train_timesteps=num_train_timesteps,
7         beta_schedule="squaredcos_cap_v2",
```

```

8         prediction_type="epsilon",
9     )
10
11 def build_ddim_scheduler(num_train_timesteps):
12     return DDIMScheduler(
13         num_train_timesteps=num_train_timesteps,
14         beta_schedule="squaredcos_cap_v2",
15         prediction_type="epsilon",
16         clip_sample=False,
17     )

```

2.3 Model Architecture

The backbone is `diffusers.UNet2DModel`. The final model uses four resolution stages with channel dimensions (128, 256, 256, 512) and two ResNet layers per block. Attention blocks are inserted at the second and fourth resolution stages. The residual blocks use `resnet_time_scale_shift="scale_shift"`, which provides FiLM-like adaptive group normalization.

For condition embedding, the final model wraps `UNet2DModel` in a `ConditionalUNet`. Instead of using a single linear projection, the 24-dimensional multi-hot label is mapped by an MLP:

$$e_y = W_2 \text{SiLU}(W_1 y).$$

The internal UNet uses `class_embed_type="identity"`, so this projected label embedding is added to the timestep embedding by the `diffusers UNet`. This is still an additive condition embedding, but the MLP gives the model more capacity than a single linear projection.

The following code shows the model wrapper. It keeps the original `diffusers UNet` implementation but replaces the default class embedding with an external label projection network.

```

1 class ConditionalUNet(nn.Module):
2     def __init__(self, num_classes=24,
3                 block_out_channels=(128, 256,
4                                     ↪ 256, 512)):
5         super().__init__()
6         time_dim = block_out_channels[0] * 4
7         self.label_proj = nn.Sequential(
8             nn.Linear(num_classes, time_dim),
9             nn.SiLU(),
10            nn.Linear(time_dim, time_dim),
11        )
12        self.unet = UNet2DModel(
13            sample_size=64,
14            in_channels=3,
15            out_channels=3,
16            block_out_channels=block_out_channels,
17            layers_per_block=2,
18            class_embed_type="identity",
19            resnet_time_scale_shift="scale_shift",
20        )
21
22 def forward(self, x_t, t, labels):
23     class_emb =
24     ↪ self.label_proj(labels.float())
25     return self.unet(x_t, t,
26                     ↪ class_labels=class_emb)
27     ↪ .sample

```

2.4 Classifier-Free Guidance

During training, I randomly drop the condition vector to zero with probability 0.1. This trains the model to handle both conditional and null conditions. During sampling, classifier-free guidance combines unconditional and conditional noise predictions:

$$\epsilon_{\text{CFG}} = \epsilon_{\text{uncond}} + w(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}),$$

where w is the CFG scale. The final model uses CFG scale 2.

The following code shows the training step. The condition dropout is the only extra operation needed for CFG training; the objective remains the standard DDPM epsilon MSE loss.

```

1 drop_mask = torch.rand(y.size(0), device=device) <
2     ↪ 0.1
3 if drop_mask.any():
4     y = y.clone()
5     y[drop_mask] = 0.0
6
7 t = torch.randint(0, 1000, (x.size(0),),
8     ↪ device=device)
9 noise = torch.randn_like(x)
10 x_t = train_scheduler.add_noise(x, noise, t)
11 eps_pred = unet(x_t, t, y)
12 loss = F.mse_loss(eps_pred, noise)

```

2.5 Classifier Guidance

The provided evaluator is a ResNet18 multi-label classifier. The specification allows using the evaluator in classifier guidance, so I inherit the evaluator class without modifying its weights or source file. Since the evaluator is trained on clean images, I do not apply it directly to noisy x_t . Instead, at each guided sampling step I estimate the clean image $\hat{x}_0$ from the current sample and predicted noise, and compute the classifier gradient on $\hat{x}_0$.

Let $f(\hat{x}_0)$ be the evaluator output probabilities. I use a positive-only guidance score:

$$S(\hat{x}_0, y) = \sum_c y_c \log f_c(\hat{x}_0).$$

The gradient $\nabla_{\hat{x}_0} S$ is used to modify the sampling direction. I found positive-only guidance to be more effective than binary cross entropy guidance, likely because the evaluator accuracy metric only checks the top- k predicted classes, where k is the number of target labels.

Classifier guidance is delayed to the latter half of sampling, from 50% to 100% of the denoising process. This avoids applying clean-image classifier gradients to very noisy intermediate samples. The final model uses CG scale 15.

The CG implementation is shown below. The sampler first forms a CFG noise prediction, estimates the clean image $\hat{x}_0$, and then uses the evaluator gradient to update the noise prediction. The gradient is applied only after the specified start ratio.

```

1 eps = eps_uncond + cfg_scale * (eps_cond -
  ↪ eps_uncond)
2
3 if cg_start <= step_id <= cg_stop:
4     x0 = predict_x0(latents, eps, scheduler, t)
5     x0 = x0.detach().requires_grad_(True)
6     prob = evaluator.resnet18(x0.clamp(-1, 1))
7     score = (labels * prob.clamp(1e-6,
  ↪ 1).log()).sum(1).mean()
8     grad = torch.autograd.grad(score, x0)[0]
9     eps = eps - cg_scale * grad.detach()
10
11 latents = scheduler.step(eps, t,
  ↪ latents).prev_sample

```

2.6 Training Configuration

The best final checkpoint was trained with AdamW, mixed precision, EMA decay 0.9999, batch size 96, and 100 epochs. I also experimented with a wider UNet, OneCycleLR, and classifier guidance during training-time evaluation. All experiments were logged with Weights & Biases.

At the end of each evaluation interval, I temporarily copy the EMA parameters into the model, generate both testing splits, compute evaluator accuracy, save image grids, and then restore the online weights. The checkpoint with the highest average of test and new_test accuracy is saved as best.pt.

3 Results

3.1 Final Accuracy

The final inference command uses the v5 no-crop checkpoint, DDIM 100 steps, seed 3, CFG scale 3, and CG scale 15:

```

1 uv run python -m scripts.sample \
2   --ckpt checkpoints_v5_nocrop/best.pt \
3   --out-dir outputs/v5_final \
4   --num-inference-steps 100 \
5   --seed 3 \
6   --guidance-scale 3 \
7   --classifier-guidance-scale 15 \
8   --classifier-guidance-start 0.5 \
9   --classifier-guidance-stop 1.0

```

Split	Accuracy
test.json	0.9722
new_test.json	0.9524

Table 1: Final evaluator accuracy of the v5 no-crop model.

Figure 1 shows the terminal screenshot of the final evaluation.

3.2 Synthetic Image Grids

The specification requires one 8×4 grid for each testing JSON file. Figure 2 and Figure 3 show the generated im-

```

(base) chueating@gpu10:/project3/chueating/DLP/Lab6$ uv run python -m scripts.sample \
--ckpt checkpoints_v5_nocrop/best.pt \
--out-dir outputs/v5_final \
--num-inference-steps 100 \
--seed 3 \
--guidance-scale 3 \
--classifier-guidance-scale 15 \
--classifier-guidance-start 0.5 \
--classifier-guidance-stop 1.0
[test] acc = 0.9722
[new_test] acc = 0.9524

```

Figure 1: Accuracy screenshot for the final model.

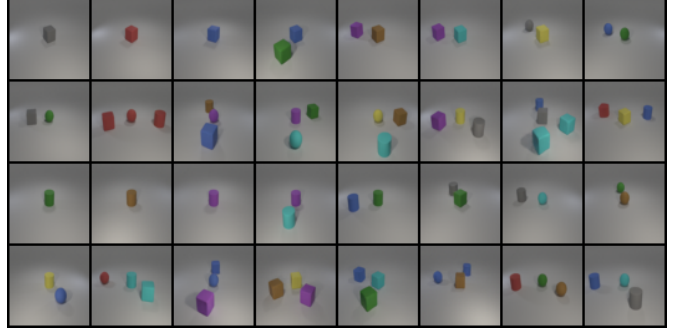


Figure 2: Generated images for test.json.

ages for test.json and new_test.json, respectively. Individual PNG files are also saved under images/test/ and images/new_test/ following the original JSON order.

3.3 Denoising Process

Figure 4 shows the denoising process for the required label set [“red sphere”, “cyan cylinder”, “cyan cube”]. The images progress from Gaussian noise to a clean object composition from left to right.

4 Experiments and Discussion

4.1 Training Dynamics

Figure 5 shows the v5 training loss and evaluator accuracy curves logged during training. The denoising loss decreases smoothly, indicating stable DDPM training. Training-time evaluator accuracy reaches roughly 0.84 average, but post-hoc inference with a stronger CFG/CG configuration on the same checkpoint pushes the average above 0.96 (Section 4.2). This gap indicates that the bottleneck at convergence is sampling guidance, not denoising quality. Earlier experiments with center-cropped inputs (v1–v4) plateaued around 0.78 even after extensive sweeps; removing the crop in v5 was the single change that broke this plateau, confirming that input coverage of off-center objects matters.

4.2 Ablation Study

The ablation study is split into two parts. Section 4.2.1 reports the systematic CFG/CG/steps sweep performed on the earlier v4 checkpoint, which used center-cropped inputs. Section 4.2.2 reports the targeted CFG/CG sweep performed on the



Figure 3: Generated images for new_test.json.

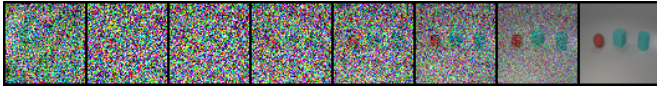


Figure 4: Denoising process for [“red sphere”, “cyan cylinder”, “cyan cube”].

final v5 no-crop checkpoint with the best configuration identified in v4.

4.2.1 v4 (center-cropped) sweep

I performed a systematic sweep on the same v4 EMA checkpoint to isolate the effect of CFG, CG, guidance loss, and sampling steps.

CFG-only sweep. Table 2 shows that the model has conditional signal, but the null path is weak. Without guidance, the accuracy is close to random. CFG improves accuracy up to scale 2–3, then saturates around 0.64 average accuracy.

CFG	Test	New-test	Avg.
0	0.0972	0.1429	0.1200
1	0.5833	0.6310	0.6072
2	0.6250	0.6548	0.6399
3	0.6250	0.6667	0.6459
5	0.6389	0.6429	0.6409

Table 2: CFG-only sweep with CG disabled.

Classifier guidance. Table 3 shows that CG gives the largest improvement. The average accuracy increases from about 0.64 with CFG only to about 0.77 with CFG+CG under seed 42. CG scales 10 and 15 are both effective; no collapse was observed under the delayed guidance setting.

Positive-only guidance vs. BCE guidance. Positive-only classifier guidance consistently outperformed BCE guidance. BCE also penalizes non-target classes, but the evaluator metric uses top- k matching rather than thresholded multi-label BCE. Therefore, emphasizing the presence of target classes is more aligned with the evaluator.

CFG	CG	Test	New-test	Avg.
1.5	10	0.6806	0.8095	0.7450
1.5	15	0.6806	0.8214	0.7510
2	10	0.7083	0.8095	0.7589
2	15	0.7083	0.8214	0.7649
3	10	0.7083	0.8333	0.7708
3	15	0.7083	0.8333	0.7708

Table 3: CFG+CG sweep with positive-only classifier guidance.

Sampling steps. I compared 50, 100, and 150 DDIM steps. 100 steps was the best trade-off. 50 steps was slightly worse, while 150 steps degraded accuracy in the tested settings, possibly because classifier guidance accumulates too strongly over more steps.

Multi-seed validation. Table 4 shows multi-seed results for the best candidate settings. The best mean setting was CFG=2 and CG=15. The best single seed was seed 3, which is used for the final output.

CFG	CG	Test	New	Avg.	Seed
3	10	0.7639	0.7690	0.7665	3
3	15	0.7778	0.7928	0.7853	3
2	15	0.7778	0.7976	0.7877	3

Table 4: Multi-seed validation on the v4 checkpoint.

4.2.2 v5 (no-crop) sweep

After removing the center crop and retraining for 100 epochs (run baseline-v5-nocrop), I re-ran the sweep with the v4-best configuration as the starting point. The training-time best checkpoint reached 0.8056 / 0.8810 on the two splits, but post-hoc inference with stronger CFG/CG dramatically improves both numbers. Table 5 sweeps CFG and CG with seed 3, the seed identified as best on v4.

CFG	CG	Test	New-test	Avg.
1.5	10	0.8750	0.9286	0.9018
1.5	15	0.8889	0.9286	0.9087
1.5	20	0.9028	0.9286	0.9157
2	10	0.9028	0.9643	0.9276
2	15	0.9444	0.9643	0.9544
2	20	0.9444	0.9643	0.9544
3	10	0.9583	0.9524	0.9554
3	15	0.9722	0.9524	0.9623
3	20	0.9722	0.9524	0.9623

Table 5: v5 no-crop CFG×CG sweep at seed 3, DDIM 100 steps, positive-only CG.

Compared with v4 in Table 3, the no-crop model gains roughly +0.20 average accuracy at every CFG/CG point. CG

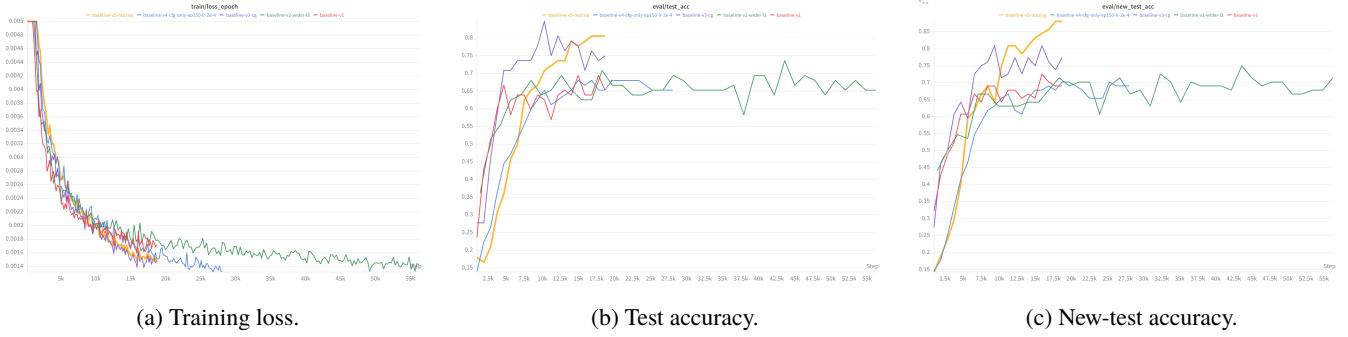


Figure 5: Training loss and evaluator accuracy curves of the v5 no-crop run logged by Weights & Biases.

is again the dominant factor, but the gains from increasing CG saturate earlier (CG 15 already matches CG 20). Increasing CFG from 2 to 3 gives the final +0.008 on average and pushes `test.json` to 0.9722.

Multi-seed validation. Table 6 shows multi-seed validation for the three top configurations. CFG=2, CG=15 is the most stable choice (highest mean over five seeds), while CFG=3, CG=15 has the highest single-seed result, which is used for the final submission.

CFG	CG	Test mean	New mean	Avg. mean
2	15	0.9000	0.9453	0.9226
3	15	0.8861	0.9262	0.9062
3	20	0.8972	0.9238	0.9117

Table 6: v5 no-crop multi-seed validation, mean over seeds 0–4.

4.3 Discussion

The experiments show that model capacity alone is not enough. The v2 wider model did not consistently outperform the v1 baseline after re-sampling, despite having more capacity and longer training. On the v4 checkpoint, the most important sampling-side improvement came from classifier guidance, which directly uses the pretrained evaluator as an image-level semantic signal. Delaying classifier guidance to the latter half of denoising is important because the evaluator is trained on clean images, not noisy diffusion states.

The single largest jump in this lab, however, came from the data side rather than the model or the sampler. Removing `CenterCrop(240)` and resizing the full 320×240 frame directly to 64×64 (v5) raised the average accuracy from about 0.79 to about 0.96 at the same CFG/CG configuration. The center crop discards roughly a quarter of the horizontal field of view, so any object placed near the left or right border is partially or fully removed from training. Once the crop is removed, the model sees all object positions present in `test.json` and `new_test.json`, and the previous plateau disappears.

The final method is still evaluator-dependent: strong classifier guidance can improve measured accuracy but may also bias images toward features recognized by the evaluator. Therefore,

I visually checked the final grids in addition to reporting the classifier accuracy.

5 Conclusion

I implemented a conditional DDPM for multi-label iCLEVR generation. The final model uses a cosine noise schedule, epsilon prediction, a label-MLP conditional UNet, EMA, DDIM sampling, classifier-free guidance, and delayed positive-only classifier guidance with the provided evaluator. Removing the center crop in the preprocessing pipeline (v5) was the single largest improvement and broke the ~ 0.78 plateau seen in earlier center-cropped runs. The final generated images achieve 0.9722 accuracy on `test.json` and 0.9524 on `new_test.json`. Ablation studies show that CFG provides conditional controllability, classifier guidance is the dominant sampling-time factor, and full-frame input coverage is the dominant data-side factor.

References

- [1] J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS, 2020.
- [2] J. Song, C. Meng, and S. Ermon. Denoising Diffusion Implicit Models. ICLR, 2021.
- [3] P. Dhariwal and A. Nichol. Diffusion Models Beat GANs on Image Synthesis. NeurIPS, 2021.
- [4] Hugging Face. Diffusers: State-of-the-art diffusion models. <https://github.com/huggingface/diffusers>.